



Um experimento sobre o uso de papéis de variáveis em Ensino de Programação Introdutória

Jorma Sajaniemi e Marja Kuittinen

Para citar este artigo: Jorma Sajaniemi & Marja Kuittinen (2005) Um experimento sobre o uso de papéis de variáveis no ensino de programação introdutória, Computer Science Education, 15:1, 59-82, DOI: 10.1080/08993400500056563

Para acessar este artigo: <https://doi.org/10.1080/08993400500056563>



Publicado online em: 16 de fevereiro de 2007.



Submeta seu artigo a esta revista. 



Visualizações do artigo: 382



Veja artigos relacionados 



Artigos que citam este texto: 6 Ver artigos que citam este texto 

Um experimento sobre o uso de papéis de variáveis no ensino de programação introdutória.

Jorma Sajaniemi* e Marja Kuittinen

Departamento de Ciência da Computação, Universidade de Joensuu, Finlândia

O conceito de "papéis das variáveis" é uma nova abordagem que captura o conhecimento tácito de especialistas de uma forma que pode ser ensinada em cursos introdutórios de programação. Um papel descreve um uso estereotipado de variáveis, e apenas dez papéis são necessários para abranger 99% de todas as variáveis em programas de nível iniciante.

Este artigo apresenta os resultados de um experimento no qual o conceito de papéis foi introduzido a alunos iniciantes em programação Pascal. Os alunos foram divididos em três grupos que receberam instruções diferentes: da maneira tradicional, sem o uso de papéis ao longo do curso; utilizando papéis durante todo o curso; e utilizando um animador de programas baseado em papéis, além do uso de papéis no ensino.

Os resultados mostram que os alunos não só conseguem compreender o conceito de papéis e aplicá-lo em novas situações, como também — e mais importante — os papéis proporcionam aos alunos uma nova estrutura conceitual que lhes permite processar mentalmente informações de programação de uma forma que demonstra boas habilidades de programação. Além disso, o uso do animador parece fomentar a assimilação do conhecimento sobre papéis.

1. INTRODUÇÃO

Habilidades de programação são necessárias desde a invenção dos computadores. Inicialmente, poucos programadores eram necessários, mas, à medida que os computadores se tornaram mais comuns, a demanda por programadores qualificados aumentou rapidamente. Naquela época, a programação era ensinada da melhor (e única) maneira conhecida, "por meio da sintaxe, utilizando uma única linguagem" (Fincher, 1999). Mais recentemente, métodos de ensino considerados eficazes foram reunidos e documentados em coleções de padrões pedagógicos (Fincher & Utting, 2002). Os novos esforços para facilitar e aprimorar o aprendizado variam em sua abordagem geral: a maioria dos estudos relata os efeitos de novos métodos de ensino e novas formas de apresentar o material didático, enquanto a reorganização de tópicos e a introdução de novos conceitos são muito mais raras.

A pesquisa sobre métodos de ensino abrange estudos que exploram o uso de diferentes maneiras de conduzir sessões de ensino, incluindo aulas expositivas combinadas com grupos de discussão.

*Autor correspondente. Jorma Sajaniemi, Departamento de Ciência da Computação, Universidade de Joensuu, PO Box 111, FI-80101 Joensuu, Finlândia. E-mail: Jorma.Sajaniemi@Joensuu.Fi

(por exemplo, Hagan & Sheard, 1998), resolução de problemas (por exemplo, Davies, 1996; Feldman, 1999; Hanly & Koffman, 1999; Koffman, 1986), observação da execução de código de exemplo ou previsão do que acontece a seguir (por exemplo, Pirolli & Anderson, 1985; Wiedenbeck, 1989) e aprendizado prático (por exemplo, Fleury, 1997; Jenkins, 1998). A outra abordagem comum, a pesquisa sobre formatos de materiais, inclui estudos que tentam explicar o efeito da apresentação de materiais de diferentes maneiras, como o uso de gráficos e metáforas gráficas em materiais de aprendizagem (por exemplo, McKay, 1999a,b) e visualização e animação de programas e algoritmos (ver Hundhausen et al. (2002) para uma visão geral). Como exemplo da terceira categoria, reorganização de tópicos, Ginat (2001) estudou as possibilidades de introduzir considerações sobre a eficiência de algoritmos em uma fase inicial do aprendizado de programação.

Conhecemos apenas dois exemplos da última categoria, pesquisa sobre novos conceitos que podem ser utilizados no ensino de programação introdutória: padrões de projeto de software e papéis das variáveis. Os padrões de projeto de software (Clancy & Linn, 1999) representam soluções independentes de linguagem e aplicação para problemas de projeto comuns. O número de padrões é potencialmente ilimitado, e existem conjuntos de padrões para vários níveis de experiência em programação (por exemplo, padrões elementares para programadores iniciantes (Wallingford, 2003)) e áreas de aplicação (por exemplo, estruturas de dados (Nguyen, 1998)). A pesquisa sobre o uso de padrões indica que os instrutores devem esperar refinar os padrões que oferecem aos alunos regularmente (Clancy & Linn, 1999).

Os papéis das variáveis (Sajaniemi, 2002, 2003) descrevem usos estereotipados de variáveis que ocorrem repetidamente em programas. Apenas dez papéis são necessários para abranger 99% de todas as variáveis em programação de nível iniciante, e eles podem ser descritos de forma concisa e facilmente compreensível (Sajaniemi, 2002). Ben-Ari e Sajaniemi (2004) demonstraram que, em uma hora de trabalho, educadores de ciência da computação podem aprender os papéis e atribuí-los com sucesso em casos normais. Ao contrário da abordagem baseada em padrões, o conjunto de papéis é tão pequeno que pode ser abordado integralmente durante um curso introdutório de programação.

Para descobrir os efeitos do uso do conceito de papéis no ensino de programação para iniciantes, realizamos um experimento em sala de aula com três condições experimentais: um grupo de alunos recebeu instrução da maneira tradicional, outro teve os papéis abordados durante o curso e o terceiro grupo utilizou papéis e animações de programas baseadas nesses papéis. Este artigo apresenta os resultados do experimento.

O objetivo do ensino é provocar alguma mudança no conhecimento e nas habilidades do aluno. Existem diversas teorias de aprendizagem com diferentes perspectivas sobre quais mudanças devem ser priorizadas e como essas mudanças podem ser alcançadas (ver, por exemplo, Hundhausen et al., 2002). O resultado da aprendizagem pode ser geralmente caracterizado como um dos seguintes: um conjunto de fatos conforme descritos nos materiais de aprendizagem; um conjunto de fatos com mecanismos de acesso eficazes (por exemplo, a teoria da dupla codificação); um conjunto de fatos autogerados (por exemplo, a teoria do construtivismo cognitivo); uma habilidade para aplicar fatos dados ou autogerados em novas situações; e, finalmente, uma replicação completa do modelo mental dos especialistas (a teoria da fidelidade epistêmica).

Programação é uma habilidade que utiliza conhecimento sobre linguagens de programação, técnicas de programação e domínio de aplicação para criar novos artefatos, ou seja, novas funcionalidades.

Assim, o objetivo do ensino de programação não pode ser apenas a introdução de um conjunto de fatos, mas também a sua aplicação em novas situações. Por outro lado, em programação, as diferenças entre novatos e especialistas são tão grandes que é irrazoável buscar fidelidade epistêmica nos primeiros cursos de programação.

Portanto, definimos como meta proporcionar aos alunos conhecimento de programação e a habilidade de aplicar esse conhecimento em novas situações, e mediremos nosso sucesso nesse nível de aprendizado.

O restante deste artigo está organizado da seguinte forma. A próxima seção descreve o conceito de papel e seus usos potenciais no ensino de programação. Em seguida, o experimento é apresentado e seus resultados são discutidos. Finalmente, a última seção contém as conclusões.

2. PAPÉIS DAS VARIÁVEIS

Sajaniemi (2002) introduziu o conceito de papéis das variáveis, que obteve como resultado de uma busca por um conjunto abrangente, porém compacto, de caracterizações de variáveis que podem ser usadas, por exemplo, para o ensino de programação e análise de programas de grande escala. Seu trabalho é baseado em estudos anteriores sobre o uso de variáveis realizados por Ehrlich e Soloway (1984), Rist (1989) e Green e Cornah (1985).

Supõe-se que os papéis capturem o conhecimento tácito especializado — uma visão apoiada pelas descobertas feitas por Ben-Ari e Sajaniemi (2004).

2.1. O Conceito de Papel

Um papel descreve o caráter dinâmico de uma variável, representado pela sucessão de valores que ela assume e como os novos valores atribuídos à variável se relacionam com outras variáveis. Por exemplo, no papel de um "stepper" (passador), uma variável recebe uma sucessão de valores que geralmente é conhecida antecipadamente, assim que a sucessão começa — embora a duração da sucessão possa ser desconhecida. O conceito de papel não se refere à forma como o valor de uma variável é usado posteriormente no programa; apenas a sucessão de valores e seus tempos de vida são relevantes.

Como exemplo, considere o programa Pascal na Figura 1. No primeiro laço, o usuário é solicitado a inserir o número de valores a serem processados posteriormente no segundo laço. O número, armazenado na variável ``data``, é solicitado repetidamente até que uma entrada válida seja obtida. A variável ``value`` é usada de forma semelhante no segundo laço: não há possibilidade de o programador adivinhar quais valores o usuário irá inserir. Como essas variáveis sempre armazenam o último valor em uma sucessão de valores, diz-se que seu papel é o de "arquivador do mais recente". A variável ``count``, no entanto, comporta-se de maneira muito diferente: uma vez inicializada, seus valores futuros serão conhecidos com exatidão. Ela diminuirá um a um até atingir seu valor limite de zero. O papel dessa variável é o de um contador.

A Tabela 1 apresenta breves descrições de todos os papéis; para uma abordagem mais abrangente, consulte a página inicial sobre Papéis de Variáveis (Sajaniemi, 2003). O organizador é o único papel especial para arrays; geralmente, o papel de um array é o de seus elementos, por exemplo, um array de coletores é ele próprio um coletor.

62 J. Sajaniemi e M. Kuittinen

```
program doubles (input, output);
var data, count, value: integer;
begin
  repeat
    write('Enter count: '); readln(data)
  until data > 0;
  count := data;
  while count > 0 do begin
    write('Enter value: '); readln(value);
    writeln('Two times ', value, ' is ', 2*value);
    count := count - 1
  end
end.
```

Figura 1. Um pequeno programa em Pascal.

Tabela 1. Definições informais de papéis.

Papel	Definição informal
Valor fixo	Uma variável que é inicializada sem nenhum cálculo e cujo valor não muda depois disso.
Stepper	Uma variável que percorre uma sucessão de valores que podem ser previsto assim que a sucessão começar.
detentor mais recente	Uma variável que armazena o último valor encontrado ao percorrer um sucessão de valores.
detentor dos mais procurados	Uma variável que contém o "melhor" valor encontrado até o momento. por meio de uma sucessão de valores. Não há restrições sobre como Medir a qualidade de um valor.
Coletor	Uma variável que acumula o efeito de valores individuais ao longo do tempo. por meio de uma sucessão de valores.
Transformação	Uma variável que sempre obtém seu novo valor a partir do mesmo cálculo. a partir do(s) valor(es) de outra(s) variável(eis).
Seguidor	Uma variável que obtém seus valores seguindo outra variável.
Bandeira de sentido único	Uma variável binária que não pode recuperar seu valor inicial depois que seu valor for alterado. foram alteradas.
Organizador	Uma matriz que é usada apenas para reorganizar seus elementos após inicialização.
Temporário	Uma variável que mantém determinado valor apenas por um período muito curto.
Outro	Qualquer outra variável.

O conjunto de funções foi obtido por meio de uma análise de todos os programas em três livros introdutórios de programação (Sajaniemi, 2002). Nesta análise, os três Os cargos mais frequentes, de valor fixo, de titular e o mais recente representaram 84% do total. variáveis.

O papel de uma variável pode mudar durante a execução de um programa e isso Isso geralmente acontece em algum lugar entre dois loops. Por exemplo, no programa de Na Figura 1, as duas variáveis, dados e contagem, poderiam ser combinadas em uma única variável.

digamos count (tornando a atribuição "count := data;" desnecessária). O papel dessa variável seria primeiro o de armazenar o valor mais recente e, em seguida, no segundo loop, o de incremento/decremento.

É importante notar que os papéis são conceitos cognitivos, e não técnicos. Como exemplo, considere a sequência de Fibonacci: 1, 1, 2, 3, 5, 8, 13, ..., onde cada número é a soma dos dois números anteriores. Um matemático que conhece bem a sequência provavelmente a enxerga com a mesma clareza que qualquer pessoa enxerga a sequência 1, 2, 3, 4, 5, ..., ou seja, o contínuo dos números naturais. Por outro lado, para um novato que nunca ouviu falar da sequência de Fibonacci e que acabou de aprender a calculá-la, cada novo número nessa sequência é uma surpresa. Portanto, o matemático pode considerar uma variável como percorrendo uma sucessão conhecida de valores (ou seja, um "passante"), enquanto o novato a considera como um "coletor", acumulando os valores anteriores para obter o próximo.

2.2. Utilizando Papéis no Ensino

O conjunto de funções é tão pequeno que pode ser totalmente abordado em um curso introdutório de programação. Como as funções são ferramentas de programação, elas não devem ser ensinadas como um tópico separado, mas sim introduzidas gradualmente à medida que aparecem nos programas. Embora exista uma definição técnica exata para cada função, definições informais (no estilo da Tabela 1) são suficientes para iniciantes.

Além do conhecimento sobre os próprios papéis, a utilização de papéis inclui o conhecimento estratégico sobre seu uso na programação. Para um iniciante, pode ser difícil começar a escrever um programa: novos conceitos de programação formam um conjunto complexo de conhecimento frágil e difícil de aplicar (Davies, 1993), e a decisão sobre qual conhecimento aplicar primeiro não é fácil. Usando o conjunto de papéis, um professor pode, no entanto, orientar os iniciantes a começar uma tarefa de programação pensando nos requisitos de dados: quais papéis (e, conseqüentemente, variáveis) são necessários para atender aos requisitos de entrada e saída da tarefa de programação e quais sequências de código são típicas para esses papéis.

O conhecimento de funções pode ser ainda mais aprimorado pela visualização e animação de programas baseados em funções. O PlanAni (Sajaniemi & Kuittinen, 2003) é um animador de programas baseado em funções que utiliza imagens de funções para visualizar variáveis e animações baseadas em funções para visualizar operações. Uma imagem de função — uma visualização usada para todas as variáveis da função — fornece pistas sobre como os valores sucessivos da variável se relacionam entre si e com outras variáveis. Por exemplo, um item muito desejado é representado por duas flores de cores diferentes: uma brilhante para o valor atual, ou seja, o melhor encontrado até o momento, e uma cinza para o valor anterior, ou seja, o segundo melhor valor.

A Figura 2 é uma captura de tela da interface do usuário do PlanAni. O painel esquerdo mostra o programa animado com um realce de cor indicando a ação atual. A parte superior do painel direito é reservada para variáveis e, abaixo dela, encontra-se a área de entrada/saída, composta por um papel para saída e uma placa para entrada. A ação ativa no painel do programa à esquerda está conectada por uma seta às variáveis correspondentes à direita. Sempre que o realce de cor é movido para um novo local no programa, o novo realce pisca.

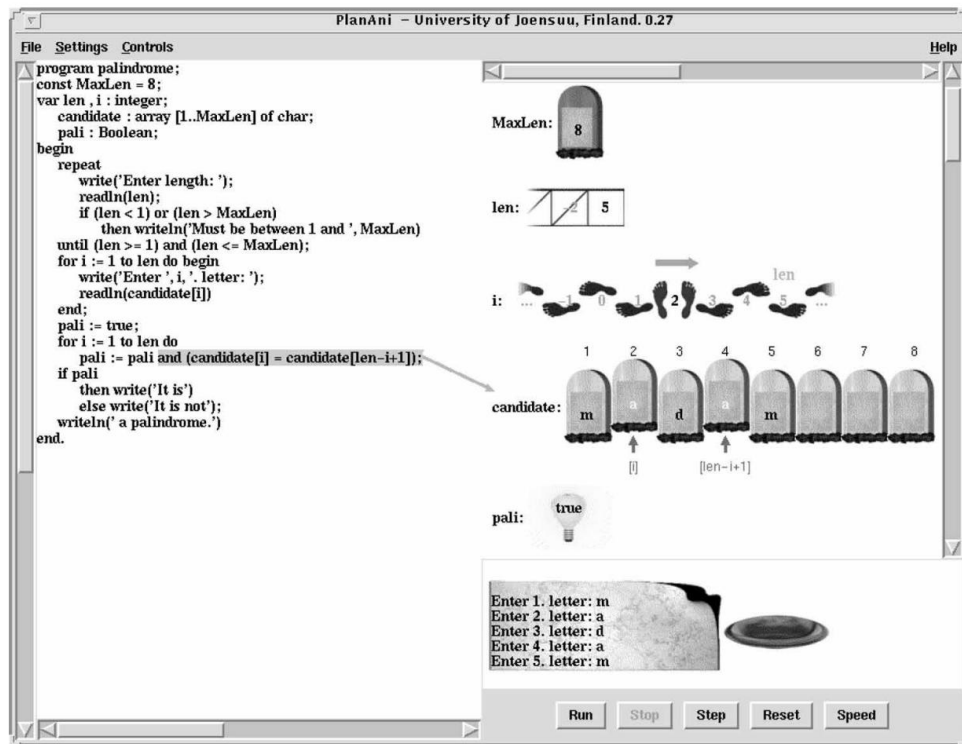


Figura 2. A interface do usuário do animador do programa PlanAni. As variáveis são representadas por imagens de funções, por exemplo, um passo (i) é representado por pegadas que compreendem os valores anterior, atual e futuro; uma matriz de valor fixo (candidato) é representada por pedras, deixando claro que os valores de seus elementos não mudarão.

3. EXPERIMENTO

Para testar a hipótese de que a introdução de funções de variáveis no ensino facilita a aprendizagem de programação, realizamos um experimento durante um curso introdutório de programação em Pascal no nível universitário. Os alunos foram divididos em três grupos que receberam instruções diferentes: da maneira tradicional como o curso havia sido ministrado diversas vezes anteriormente, ou seja, sem tratamento específico de funções; utilizando funções ao longo de todo o curso; e utilizando um animador de programas baseado em funções nos exercícios, além do uso de funções no ensino. O curso teve duração de cinco semanas, com quatro horas de aulas teóricas e duas horas de exercícios por semana.

Ao final do curso, foi aplicada uma prova que foi corrigida normalmente para os fins do curso. As respostas dos alunos, no entanto, foram analisadas para este experimento de outras maneiras, a fim de encontrar diferenças qualitativas entre os grupos.

Para evitar que os alunos trocassem de turma constantemente, as aulas foram agendadas para o mesmo horário. Consequentemente, foi necessário utilizar dois professores. Ambos os professores tinham vasta experiência em ministrar aulas para alunos de graduação e já haviam lecionado a disciplina anteriormente. O professor responsável pela aula era o professor que ministrava as aulas.

As aulas tradicionais não estavam familiarizadas com o conceito de papéis. Portanto, não houve efeitos negativos do professor evitar certos tópicos em suas aulas, visto que ele não sabia exatamente do que se tratava o experimento. Para detectar quaisquer diferenças causadas por diferentes professores e diferentes níveis de engajamento dos alunos no curso, a prova incluiu questões de controle que não estavam relacionadas às variáveis e, portanto, esperava-se que produzissem resultados semelhantes em todos os grupos.

Tanto no meio quanto no final do curso, alguns alunos receberam tarefas de compreensão e construção de programas, que foram gravadas em vídeo. Esses protocolos serão analisados posteriormente para identificar diferenças qualitativas no nível conceitual das expressões utilizadas pelos alunos ao falar sobre programas.

3.1. Método

O experimento teve um delineamento entre sujeitos, com o conteúdo da instrução como fator entre sujeitos. Os participantes foram divididos em três grupos: um que recebeu aulas e exercícios normais (grupo tradicional), um que assistiu a aulas com uso sistemático de papéis variáveis ao longo do curso (grupo de papéis) e um que assistiu às mesmas aulas do grupo de papéis, mas com o uso de um animador baseado em papéis nos exercícios (grupo de animação).

Todos os grupos receberam os mesmos materiais didáticos e programas de exemplo, com a única exceção sendo a apresentação dos papéis. Nos grupos de papéis e animação, os papéis foram introduzidos gradualmente nas aulas, à medida que apareciam nos programas de exemplo. No material de apoio das aulas, as declarações de variáveis eram acompanhadas de seus respectivos papéis em comentários. Os alunos também receberam uma lista impressa descrevendo todos os papéis com exemplos de programas curtos (4 páginas). Nos exercícios, o papel de cada variável era mencionado aos alunos. No grupo tradicional, o mesmo tempo de aula foi dedicado, sem a menção explícita dos papéis. Os materiais de apoio eram equivalentes aos dos outros grupos, exceto pela ausência dos nomes dos papéis nos comentários das variáveis. Como substituto da lista de papéis, os alunos do grupo tradicional receberam os mesmos programas como "exemplos adicionais". Durante as aulas, os mesmos programas foram explicados para todos os grupos. As aulas foram baseadas em materiais existentes, não especificamente elaborados para a introdução de papéis; essa decisão foi tomada com a intenção de não interferir no ensino do grupo tradicional e de apresentar a todos os grupos um ensino o mais semelhante possível.

Durante os exercícios, todos os grupos executaram quatro programas; um programa em cada sessão de exercícios, exceto na primeira. Nessas tarefas, o grupo de animação utilizou o animador de programas baseado em papéis PlanAni, enquanto os outros grupos utilizaram um depurador visual (Turbo Pascal v. 7.0). Cada sessão de exercícios começou com os alunos apresentando suas soluções para as tarefas de casa. As animações, com duração entre 20 e 40 minutos, foram sempre apresentadas ao final das sessões. Em cada sessão, a professora apresentou a animação passo a passo, utilizando seu computador e um projetor de vídeo. No grupo de animação, a professora explicou, para cada novo papel, qual era a imagem do papel e como ela tentava visualizar as propriedades mais importantes do papel. Em todos os grupos, os alunos foram então instruídos a executar a animação utilizando os dados fornecidos, cuidadosamente selecionados pela professora.

Em seguida, os alunos animaram o programa com seus próprios dados de entrada. Finalmente, o professor discutiu com os alunos as questões complexas ou outras dificuldades que eles tiveram para entender o programa. Durante todo o processo, os alunos foram incentivados a prosseguir lentamente com a animação e a prever o efeito da próxima instrução sobre os valores das variáveis e outros aspectos do programa.

Hundhausen et al. (2002) argumentam que a forma como os alunos utilizam a tecnologia de visualização tem um impacto maior na eficácia do que o conteúdo das visualizações. Ao utilizarmos as mesmas tarefas e atividades em todos os grupos, procuramos garantir que as atividades cognitivas fossem equivalentes em cada grupo, de modo que as diferenças de desempenho não fossem devidas a diferenças nas atividades cognitivas, mas sim a diferenças no conteúdo das visualizações.

3.2. Assuntos

Os participantes do estudo foram alunos de graduação do primeiro semestre do curso de Ciência da Computação. Todos os alunos assistiram à mesma primeira aula, onde responderam a um breve questionário com informações sobre suas notas do ensino médio e sua experiência prévia com computadores e programação. Após a primeira aula, os alunos ($n = 80$) foram divididos aleatoriamente em três grupos, e testes qui-quadrado foram realizados para verificar diferenças estatisticamente significativas entre os grupos em relação às notas e à experiência. Esse procedimento foi repetido até que grupos sem diferenças significativas fossem encontrados e as médias do grupo tradicional fossem iguais ou superiores às médias dos outros grupos para as propriedades que consideramos mais importantes para prever o aprendizado de programação. Wilson e Shrock (2001) estudaram diversos fatores e descobriram que o nível de conforto, a formação em matemática e o treinamento formal em programação influenciam positivamente o sucesso em um curso introdutório de programação. O nível de conforto não pode ser mensurado no início de um curso e, em nosso caso, os cursos de programação anteriores eram praticamente os mesmos para todos os alunos. Portanto, selecionamos o conhecimento de matemática do ensino médio, a criação de planilhas e a experiência em programação como as propriedades mais importantes. A maior diferença entre os grupos foi na experiência em programação ($w_2 = 4,054$, $df = 2$, $p = 0,3988$).

Após a primeira aula, 11 novos alunos se matricularam. Devido aos limites de tempo rigorosos, eles não puderam ser alocados usando este procedimento. No entanto, os grupos ainda mantiveram sua adequação para o experimento, conforme mostrado na Tabela 2, que resume as principais características dos grupos. A última coluna da tabela apresenta os valores de p dos testes χ^2 e indica que não houve diferenças estatisticamente significativas entre os três grupos.

Na criação de planilhas, o valor 1 corresponde a um curso introdutório que todos os novos alunos devem fazer no início de seus estudos. Em cursos de programação, o valor 1 corresponde a uma breve introdução opcional à programação que precede o curso experimental. Essa breve introdução utiliza a linguagem Karel, que não possui variáveis. Em experiência em programação, o valor 1 corresponde a ter escrito alguns pequenos programas usando alguma linguagem de programação com variáveis; por exemplo, o uso de Karel ou HTML não foi considerado como programação.

Tabela 2. Dados básicos sobre os grupos experimentais. Em todas as escalas, valores mais altos são melhores.

Propriedade	Grupo				p
	Tradicional	Animação de Papéis		Todos	
Número de disciplinas	26	32	33	91	
Sujeitos do sexo feminino (%)	30,8	18,9	24.2	24.2	0,7380
Matemática do ensino médio (média, escala 1 – 3)	2.4	2.2	2.1	2.2	0,1343
Língua materna do ensino médio (média, escala 1 – 3)	2.4	2.2	2.1	2.2	0,1851
Tecnologia da informação do ensino médio (média, escala de 1 a 3)	2,5	2.6	2.6	2.6	0,9828
Arte do ensino médio (média, escala de 1 a 3)	2.2	2.3	2.4	2.3	0,7039
Criação de planilha (média, escala 0 – 2)	1.3	1.0	1.2	1.1	0,2827
Cursos de programação (média, escala 0 – 2)	0,8	0,9	1.0	0,9	0,9539
Experiência em programação (média, escala 0 – 2)	0,8	0,5	0,8	0,7	0,4566

3.3. Materiais

O exame consistia em quatro tipos de questões (o número de questões em parênteses):

- Questões de controle não relacionadas a variáveis (E-NONVAR, 2): Estas foram utilizadas para identificar possíveis diferenças entre os professores e fornecer um ponto de referência. para cada disciplina, refletindo suas capacidades pessoais e quantidade de envolvimento no curso. Na análise dos resultados, essas questões foram utilizadas como um "pré-teste" para avaliar as pontuações de outras questões. Para esse fim, estes As perguntas foram elaboradas para testar um tipo de aprendizagem semelhante ao do experimento. perguntas, ou seja, uma habilidade para aplicar o material aprendido em novas situações. A primeira questão dizia respeito a várias estruturas de repetição e situações para as quais Cada uma delas é apropriada. A segunda questão apresentou regras sintáticas para um linguagem estranha juntamente com possíveis sequências da linguagem. Os sujeitos eram Perguntaram quais cordas eram legais e por quê.
- Simulação de programa (E-SIMU, 1): Os participantes foram solicitados a prever a saída de um Programa de 15 linhas com dados de entrada fornecidos. O programa encontrou números primos. números usando o crivo de Eratóstenes e seu resultado continham os números primos juntamente com a soma acumulada.
- O programa continha dois controladores de passo, um controlador de valor fixo, um coletor e um controlador unidirecional. Matriz de flags; os nomes das variáveis são identificadores de uma letra sem significado.
- O uso de funções era claro, mas a lógica do programa deveria ser... complicado (promovido pelos nomes de variáveis sem sentido) para que os alunos Seria necessário usar simulação para decifrar a saída do programa.

Compreensão do programa (E-COMPR, 1): Os participantes receberam um programa de 19 linhas que imprimia uma tabela de dosagem para uma semana de medicação, juntamente com a quantidade total de medicamento necessária. A tarefa dos alunos era "descrever qual é o propósito do programa e como ele funciona", ou seja, escrever um resumo do programa. O programa tinha um valor fixo, um incremento, um detentor do valor mais recente e um coletor. As variáveis eram letras únicas com significado, exceto a única variável de entrada (o peso do paciente), que era uma palavra completa com significado.

O programa tinha uma lógica simples e um domínio facilmente compreensível. Esperávamos que praticamente todos os alunos entendessem o programa e estávamos interessados em analisar as maneiras como eles o explicariam.

As variáveis foram nomeadas de forma significativa para promover o reconhecimento do domínio e facilitar a compreensão. Identificadores com palavras completas foram evitados para possibilitar a distinção entre nomes de variáveis e conceitos do domínio na análise de resumos de programas.

Construção do programa (E-CONSTR, 1): Os participantes foram solicitados a escrever um programa que, inicialmente, recebesse como entrada o número de sessões de exercícios e o número total de tarefas de exercícios. Em seguida, o número de tarefas concluídas em cada sessão de exercícios para um aluno seria fornecido como entrada, e o programa deveria calcular se o aluno concluiu o número necessário de tarefas. Esse processo seria repetido tantas vezes quanto o número de alunos.

Esta tarefa de programação foi concebida para ser compreensível para os alunos que farão o exame e para exigir o uso de várias funções. Uma solução ótima utilizaria dois detentores de valores mais recentes que mudariam para valores fixos após a fase inicial do programa, dois incrementadores, um detentor de valores mais recentes e um coletor.

Além disso, os participantes foram solicitados a relatar o quão ativamente haviam comparecido às aulas e aos exercícios. Os alunos que compareceram a menos de 40% das aulas ou exercícios foram...

Foram excluídos dos resultados porque o efeito da instrução sobre seu desempenho foi questionável.

Os materiais experimentais podem ser encontrados em http://www.cs.joensuu.fi/*saja/var_roles/literatura.html.

3.4. Procedimento

O exame, que era um requisito do curso, teve duração de quatro horas.

As respostas dos alunos foram inicialmente avaliadas normalmente e, em seguida, analisadas para os fins deste experimento. A nota máxima era de 6 pontos para cada questão. Todas as notas foram verificadas por outro professor. Embora vários professores tenham participado da avaliação, todas as respostas de cada questão foram avaliadas pelos mesmos alunos do primeiro e segundo ano. Uma análise mais aprofundada dos resultados foi realizada pelos autores. Nem os avaliadores nem os pesquisadores responsáveis pela análise sabiam a qual grupo os alunos pertenciam.

3.5. Resultados

Sessenta alunos compareceram ao exame. Os alunos que compareceram a menos de 40% dos exames foram desclassificados. Palestras ou exercícios foram descartados, restando 44 disciplinas para a análise. A Tabela 3 lista os resultados. Médias e desvios padrão para cada questão e cada grupo. Diferenças
As diferenças entre os grupos não são significativas para nenhuma das questões.

3.5.1. Questões de Controle As notas das duas questões E-NONVAR se comportam Da mesma forma: o grupo tradicional é o melhor, o grupo de animação é o segundo melhor, e assim por diante. grupo com pior desempenho em ambas as áreas. Coeficiente de correlação de Pearson entre as duas notas. é $r = 0,412$, a probabilidade bicaudal de ocorrência de uma correlação dessa magnitude por chance sendo estatisticamente significativa ($SE(b) = 0,281$, $t = 2,931$, $df = 42$, $p = 0,0054$).

Essas duas questões não estavam relacionadas às variáveis de forma alguma; portanto, as diferenças em As notas não dependem da variável independente — o conteúdo da instrução — mas refletem variáveis que não puderam ser controladas: diferenças entre professores, e nível de envolvimento dos participantes no curso. Para compensar essas diferenças, nós Não utilizaremos as notas da Tabela 3 em si, mas sim a diferença entre um a nota do aluno (E-SIMU, E-COMPR, E-CONSTR) e sua média nas duas disciplinas. Classificações E-NONVAR como pontuações para análises posteriores. Para facilitar a visualização dos dados. Além disso, ajustaremos as diferenças para que a média das pontuações do O grupo tradicional terá como média a nota original.

Essas diferenças entre os grupos também devem ser levadas em consideração quando Considerando resultados que não podem ser compensados dessa maneira simples. Por exemplo, proporções percentuais de vários tipos de declarações em resumos de programas Não podem ser compensados, e é preciso ter cautela ao interpretar os resultados.

3.5.2. Simulação do Programa As pontuações para a questão de simulação do programa (E-SIMU) são Apresentado na Figura 3. As diferenças entre os grupos não são significativas.

Tabela 3. Notas originais no experimento.

Pergunta	Grupo					
	Tradicional n = 10		Funções n = 17		Animação n = 17	
	Significar	SD	Significar	SD	Significar	SD
E-NONVAR/1	4,9	0,98	3,9	1,69	4,6	1,48
E-NONVAR/2	4,3	1,11	3,5	1,61	3,9	1,56
E-SIM	3,4	2,21	2,5	2,20	2,8	2,36
E-COMPR	4,5	0,59	4,1	1,69	3,6	1,52
CONSTRUÇÃO ELETRÔNICA	3,8	1,48	3,8	1,72	3,9	1,51

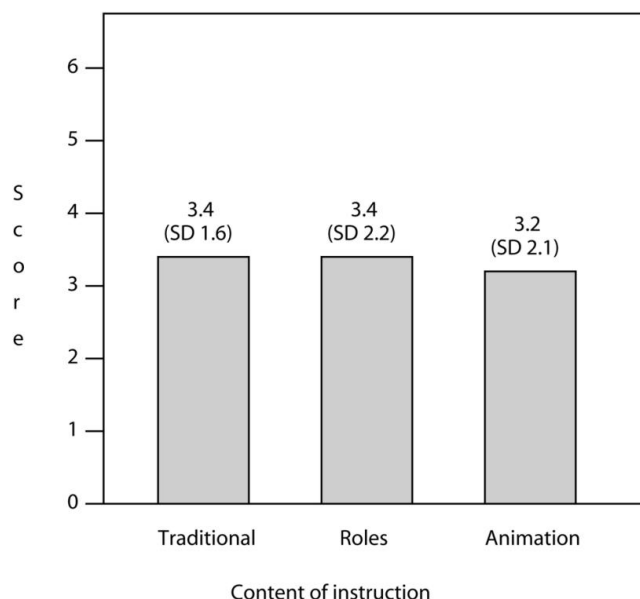


Figura 3. Pontuações médias (e desvios padrão) da questão de simulação do programa (E-SIMU). As pontuações são obtidas compensando as notas originais com as notas das questões de controle.

3.5.3. Compreensão do Programa Os resultados da questão de compreensão do programa (E-COMPR) são apresentados na Figura 4. A diferença entre os grupos de papéis e animação é significativa (teste t bicaudal, $t = 2,026$, $gl = 41$, $p = 0,0493$).

Para estudar essa diferença, as respostas foram analisadas em relação à correção da compreensão. Seleccionamos todas as respostas sem erros e que demonstravam pleno entendimento de cada detalhe do programa, e calculamos as médias das notas para cada grupo. Para o grupo tradicional, a média foi de 4,6; para o grupo de papéis, 5,0; e para o grupo de animação, 4,1 (papéis vs. animação, teste t bicaudal, $t = 1,718$, $gl = 27$, $p = 0,0972$). Na prova, as respostas foram avaliadas não apenas com base na completude da compreensão, mas também na "qualidade" — conforme percebida pelos professores responsáveis pela avaliação — do resumo. Como todas as respostas seleccionadas para a nova análise demonstraram pleno entendimento do programa, as diferenças nas médias das notas implicam diferenças na forma como os participantes descreveram o programa.

A literatura anterior (por exemplo, Pennington, 1987; Good, 1999) indica que as diferenças qualitativas nos resumos de programas estão relacionadas à qualidade da compreensão. Portanto, analisamos todos os resumos de programas (incluindo aqueles com erros) mais detalhadamente. Para essa análise, utilizamos o esquema de análise de resumos de programas de Good (Good, 1999), que consiste em duas classificações: uma baseada em tipos de informação e a outra baseada em descrições de objetos. A classificação por tipos de informação é um refinamento mais detalhado e totalmente especificado do esquema de Pennington (Pennington, 1987), enquanto a classificação por objetos é uma versão mais restrita do nível de detalhamento de Pennington. Pennington utilizou seu esquema com especialistas trabalhando em um programa de porte moderado, enquanto nós

Os participantes eram novatos trabalhando em um programa curto — um grupo semelhante ao de Good. assuntos.

A classificação dos tipos de informação é usada para codificar declarações resumidas sobre o com base nos tipos de informação que contêm. A Tabela 4 contém a distribuição de Declarações sobre o tipo de informação para cada um dos grupos experimentais. (Um sujeito de cada grupo)

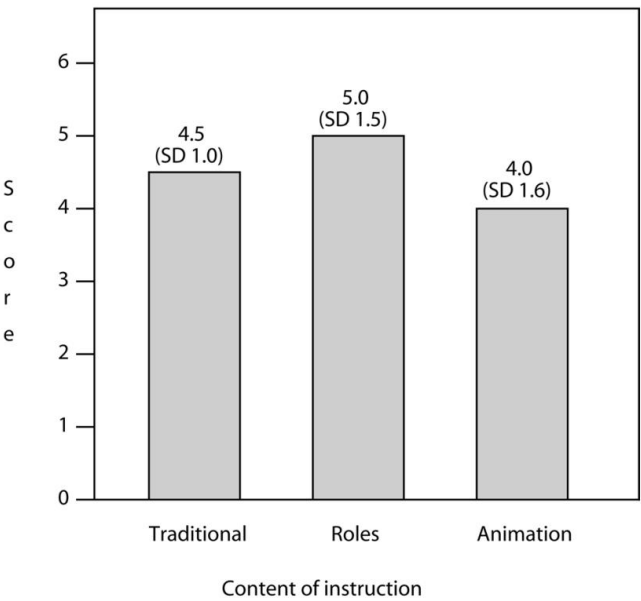


Figura 4. Pontuações médias (e desvios padrão) da questão de compreensão do programa. (E-COMPR).

Tabela 4. Percentagens médias de declarações do tipo informação nas respostas de compreensão do programa (E-COMPR).

Tipo de informação	Grupo		
	Tradicional	Funções	Animação
Função	10.1	8.2	9,6
Ações	20,4	18.1	21.2
Operações	11.9	13.2	11.9
Alto nível estadual	0,4	2.1	0,0
Estado-baixo	0,2	0,6	0,5
Dados	26,9	25.3	30,9
Controlar	6.0	6.0	6.0
Elaborar	22.3	26.1	18,6
Meta	2.0	0,3	1.3

O grupo de funções foi excluído desta análise: ele estava principalmente copiando o código do programa e seu comportamento era muito diferente de qualquer outro sujeito que ameaçasse a análise estatística.) Para definições detalhadas dos tipos de informação, veja (Good, 1999).

As médias foram testadas usando ANOVA; a única diferença significativa foi no estado-alto ($F(2,40) = 3,028$, $p = 0,0596$), que continha declarações resumidas descrevendo o estado atual do programa em um nível mais abstrato do que os simples resultados dos testes. O grupo de papéis usou mais declarações de estado-alto do que os outros grupos, mas o número absoluto de declarações foi pequeno. No grupo de papéis, 31,3% dos participantes usaram declarações de estado-alto, enquanto no grupo tradicional apenas 10,0% dos participantes as usaram, e no grupo de animação nenhum participante as usou.

A maior diferença absoluta foi nas declarações elaboradas, ou seja, informações adicionais sobre algo que já foi descrito. O grupo de papéis usou mais declarações elaboradas do que os outros grupos, mas essa diferença não é estatisticamente significativa ($F(2,40) = 1,032$, $p = 0,3654$).

A outra classificação de Good, a descrição de objetos, analisa a maneira como os objetos são descritas. Cada referência a um objeto é classificada como uma das seguintes:

. somente no programa: referência a um item que ocorre apenas no domínio do programa, por exemplo, "contador"

. programa: referência descrita em termos de programação, por exemplo, um identificador de variável. . programa – mundo real: referência que usa terminologia válida tanto no mundo real quanto no domínio da programação, por exemplo, "número". .

programa – domínio: uma mistura de referências de domínio de programação e de domínio do problema, por exemplo, "lista de alturas".

Domínio: referência usando termos do domínio, por exemplo, "dias ensolarados".

Referência indireta: uma referência anafórica, por exemplo, "isso".

Inconclusivo: uma referência ambígua que não pode ser codificada.

A Tabela 5 contém a distribuição das descrições dos objetos para cada um dos grupos experimentais.

Os grupos de papéis e animação usaram mais instruções somente de programa ($F(2,40) = 1,791$, $p = 0,1800$; teste t bicaudal tradicional vs. animação, $t = 1,798$, $gl = 40$, $p = 0,0797$), principalmente porque possuíam um vocabulário rico para nomes de papéis, em oposição ao vocabulário do grupo tradicional, que consistia apenas em "contador" e "contador de loop". Por outro lado, as instruções de programa foram as menos comuns no grupo de animação, mas essa diferença não é estatisticamente significativa ($F(2,40) = 1,012$, $p = 0,3725$).

Embora as médias dos grupos na Tabela 5 sejam semelhantes, houve diferenças significativas na distribuição das declarações dentro dos grupos: os resumos de programas no grupo tradicional apresentavam um número pequeno ou grande de declarações de domínio, enquanto nos outros dois grupos as declarações de domínio foram utilizadas de forma mais uniforme. Para analisar melhor essa diferença, utilizamos uma estratégia semelhante à de Pennington (1987) e classificamos os resumos de programas em três tipos, dependendo da quantidade de declarações de domínio em relação às declarações de programa.

Tabela 5. Percentagens médias de declarações de descrição de objetos nas respostas de compreensão de programas. (E-COMPR).

Tipo de descrição do objeto	Grupo		
	Tradicional	Funções	Animação
Somente o programa	0,8	2,4	5,1
Programa	27,1	24,8	16,0
Programa – Mundo real	0,2	1,3	0,5
Programa – Domínio	11,3	11,9	9,6
Domínio	49,9	52,6	55,2
Referência indireta	5,1	3,0	4,7
Não está claro	5,7	3,9	9,0

Declarações em descrições de objetos. Resumos com pelo menos 67% de declarações de domínio. (Excluindo declarações indiretas e pouco claras) foram denominados resumos de nível de domínio. Resumos com pelo menos 67% de declarações referentes apenas a programas foram classificados como resumos em nível de programa, e todos os outros foram chamados de resumos com referências cruzadas, porque tinham uma distribuição mais uniforme de informações sobre domínio e programa.

A Figura 5 ilustra a distribuição dos participantes em cada grupo com base no tipo de...

Eles forneceram um resumo do programa. O número de resumos com referências cruzadas foi significativamente menor no grupo tradicional do que nos outros grupos.

($w^2 = 10,773$, $df = 2$, $p = 0,0046$). No estudo de Pennington, alta compreensão

Os programadores usavam, quase unanimemente, resumos com referências cruzadas, enquanto os menos

Os programadores de compreensão tendiam a produzir um resumo em nível de programa.

ou um resumo em nível de domínio. Assim, os resumos do programa descrevem as funções e a animação.

Os grupos eram semelhantes aos de programadores com alto nível de compreensão.

Essa análise, no entanto, não explica as diferenças nas notas de acertos.

respostas. Para estudar isso mais a fundo, analisamos a correlação entre notas e

Resumo do programa: propriedades das respostas que demonstram completude ou quase completude.

compreensão. Além do tipo de informação e do conteúdo da descrição do objeto,

O comprimento e a exatidão do resumo também foram incluídos nesta análise. (Comprimento do resumo)

A média foi de 152,1 palavras para o grupo tradicional, 154,4 para o grupo de papéis, e

141,4 para o grupo de animação.)

A Tabela 6 lista todas as correlações estatisticamente significativas. As maiores correlações são

Resumos negativos e preocupantes de características gerais de programas: resumos com um

Uma maior proporção de declarações de ação e terminologia de domínio resultou em notas piores.

enquanto resumos com instruções de operação de nível inferior e terminologia de programa foram

melhores notas. Até mesmo a extensão dos resumos e a quantidade de repetição nas respostas.

correlacionam-se mais com a nota do que com a correção dos resumos.

A última coluna da Tabela 6 indica se o grupo de funções ou o grupo de animação pertence ao grupo de papéis.

teve uma média maior para a propriedade em questão. É interessante notar que, por quase

Todas as propriedades apresentaram correlação positiva com a nota, e o grupo de funções teve um valor maior.

Tabela 6. Correlações de Pearson entre as notas de Compreensão do Programa (E-COMPR) e o Programa Propriedades resumidas. A última coluna indica se a média do grupo de funções foi menor. ('5'), maior ('4') ou igual (" = ") ao do grupo de animação

Resumo da propriedade	Correlação	p	Papéis versus animação
Tipo de informação / Ações	7 0,763	0,000	5
Descrição do objeto / Domínio	7 0,731	0,000	5
Resumo do comprimento	+ 0,629	0,000	4
Descrição do objeto / Programa	+ 0,548	0,001	4
Tipo de informação / Detalhado	+ 0,544	0,001	4
Descrição do objeto / Não clara	+ 0,414	0,012	5
Correção do resumo	+ 0,413	0,012	=
Tipo de informação / Operações	+ 0,395	0,017	4

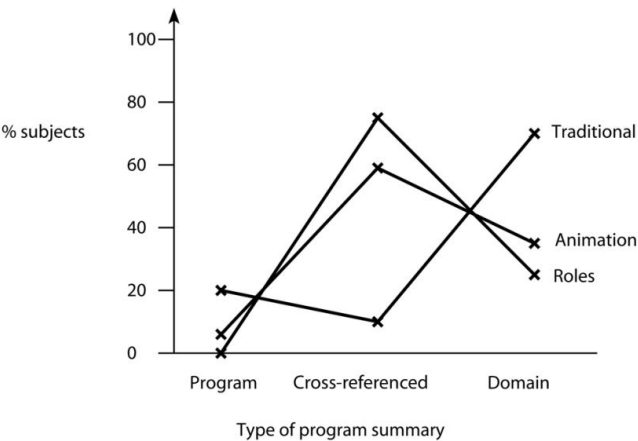


Figura 5. Distribuição de assuntos com diferentes tipos de resumo do programa (E-COMPR). Tipo de resumo é determinado pela quantidade de informações sobre domínio e programa nas descrições dos objetos.

média do que o grupo de animação, e vice-versa. Mesmo que a correlação não
Em termos de causalidade média, as diferenças nas notas agora são fáceis de explicar: atribuição de notas pelos professores.
Os resumos privilegiavam propriedades de resumo que eram mais comuns entre os papéis.
grupo (ou seja, informações de baixo nível e talvez proximidade) e propriedades desfavoráveis
que eram mais comuns no grupo de animação (ou seja, informações de alto nível).

3.5.4. Construção do Programa Os resultados da questão de construção do programa (E-CONSTR) são
apresentados na Figura 6. As diferenças entre os grupos não são significativas. Para obter uma melhor
compreensão das possíveis diferenças entre
Os grupos em que classificamos os erros (excluindo pequenos erros sintáticos) foram de diversas maneiras.
Em alguns programas, houve grandes omissões e foi impossível contabilizar os participantes.
número de erros individuais. Portanto, não pudemos basear nossa análise em taxas de erro.

Em vez disso, analisamos se os participantes apresentavam problemas em determinados tipos de erros e Contou o número de indivíduos com problemas.

A primeira classificação de erros considera o tipo de conhecimento faltante que seria necessário corrigir o erro. Existem três categorias: linguagem de programação conhecimento, ou seja, problemas com a sintaxe ou semântica de Pascal, programação conhecimento, por exemplo, problemas com loops aninhados, e conhecimento do domínio, ou seja, problemas com compreensão dos objetivos do programa ou em matemática elementar. Tabela 7 Contém as porcentagens de indivíduos que cometeram erros de cada tipo. Apenas alguns indivíduos apresentaram erros. problemas com a linguagem de programação, mas quase todos cometeram erros de programação. e teve dificuldades em compreender o problema de programação em detalhes. Diferenças As diferenças entre os grupos são estatisticamente insignificantes.

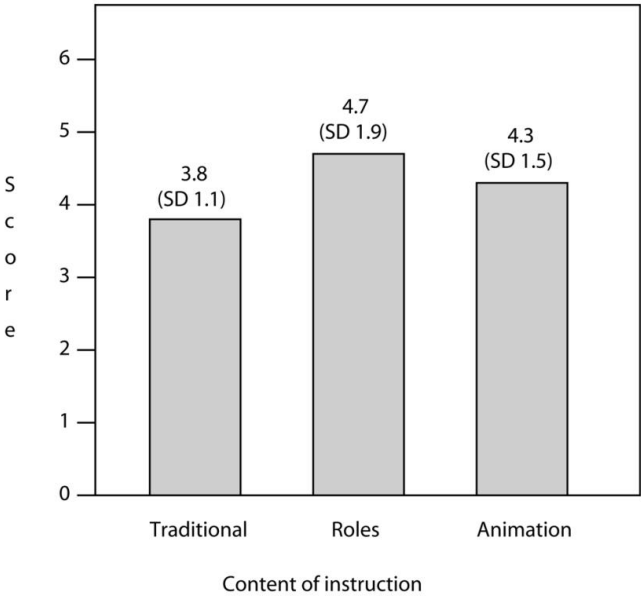


Figura 6. Pontuações médias (e desvios padrão) da questão de construção do programa (E-CONSTR).

Tabela 7. Porcentagens de sujeitos que cometeram erros na construção de programas (E-CONSTR) ordenadas por Tipo de conhecimento necessário.

Tipo de conhecimento necessário	Grupo		
	Tradicional	Funções	Animação
Linguagem de programação	20,0	5,9	5,9
Programação	90,0	94,1	94,1
Domínio	80,0	88,2	70,6

Como os métodos de ensino não deveriam afetar a compreensão do domínio, mas sim o conhecimento de programação, analisamos os erros de programação com mais detalhes.

Em particular, estávamos interessados em descobrir em qual nível de conhecimento de programação os participantes encontravam dificuldades ao construir seus programas. Pennington (1987) constatou que o conhecimento de programação referente a operações e estruturas de controle reflete um conhecimento superficial, ou seja, um conhecimento facilmente acessível pela simples observação do programa. Em contrapartida, o conhecimento referente ao fluxo de dados e à função do programa reflete um conhecimento profundo, o que indica uma melhor compreensão do código. Classificamos os erros de programação em três categorias, começando pelo nível superficial e progredindo para casos mais complexos que exigem um conhecimento mais aprofundado das diversas interações dentro de um programa:

. controle: erros no uso de estruturas de controle. . dados: erros

relacionados ao fluxo de dados dentro e entre variáveis. . dados especiais: erros

relacionados ao uso de variáveis em condições excepcionais, por exemplo, valores de entrada sentinela que não devem ser processados.

A Figura 7 mostra que, embora o grupo tradicional tenha apresentado o melhor desempenho e o grupo de animação o pior no que diz respeito à estrutura superficial, o oposto ocorre em níveis mais profundos de conhecimento do programa. A maior diferença é observada entre o grupo tradicional e o grupo de animação no caso de dados especiais ($w^2 = 2,830$, $gl = 1$, $p = 0,0998$).

Também analisamos todos os erros relacionados às variáveis, observando as funções das variáveis e o tipo de erro. A Tabela 8 lista todos os tipos de erro classificados por função e complexidade de uso. As maiores diferenças dizem respeito ao uso complexo do stepper, com o grupo de funções apresentando o pior desempenho, e ao uso complexo do holder mais recente, com o grupo de animação apresentando o melhor desempenho. (Todas as diferenças entre os grupos são estatisticamente insignificantes.)

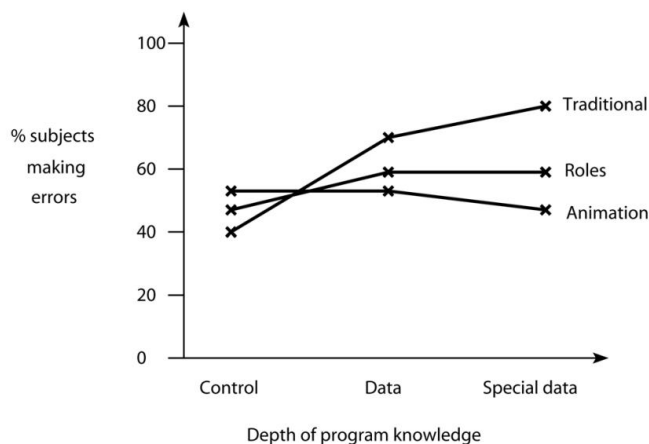


Figura 7. Percentagens de participantes que cometeram erros de programação na construção de programas (E-CONSTR), ordenadas por nível de conhecimento de programação.

As possíveis origens desses erros são diversas. Por exemplo, inicializações mal posicionadas e As atualizações indicam problemas com as estruturas de controle, enquanto o uso desnecessário de um A matriz indica conhecimento de plano ausente (ou frágil (Perkins & Martin, 1986)) (De' tienne, 2002), por exemplo, o sujeito não sabe como usar um detentor mais recente para percorrer uma sucessão de valores. Da mesma forma, uma inicialização ausente significa que O sujeito utilizou a expansão focal (Rist, 1989), ou seja, escreveu a atualização antes. inicialização, o que indica falta de conhecimento do plano.

Por fim, as atualizações ausentes precisam de mais análise, pois constituem a linha focal. de um plano variável (Rist, 1989) e não se espera que sejam perdidos. Neste programa, A ausência de atualizações de um detentor mais recente simples (caso (a) na Tabela 8) eram problemas com compreensão do domínio da tarefa e, no caso (c), havia problemas com o controle. estruturas. No caso (b), não há explicação evidente; a maior frequência de erro Estava, no entanto, entre o grupo de papéis que obteve as notas mais baixas no grupo de controle. perguntas. Pode-se questionar se a tarefa de construção do programa era tão... exigindo que suas habilidades justificassem a omissão como sendo causada pela memória de trabalho. sobrecarga.

A Tabela 9 resume as possíveis origens de erros no uso simples e complexo. contexto. Todos os grupos tiveram problemas com estruturas de controle, tanto em contextos simples quanto complexos. contextos complexos. O grupo de animação demonstrou conhecimento do melhor plano: eles Possuíam os planos variáveis necessários (ou seja, funções) e os aplicaram corretamente. Finalmente, Tanto o grupo de funções quanto o grupo de animação tiveram problemas com o domínio. compreensão — um fato que não podemos explicar.

3.5.5. Uso de Nomes de Cargos Por fim, analisamos o uso de nomes de cargos nas respostas. Como Como esperado, nenhum participante do grupo tradicional utilizou nomes de funções. As funções e O grupo de animação comportou-se de forma semelhante: 35% dos participantes em ambos os grupos utilizaram o papel. nomes. Os papéis geralmente eram atribuídos corretamente, com apenas dois erros cometidos. sujeitos no grupo de funções.

Tabela 8. Erros relacionados às variáveis. Os números representam a porcentagem de participantes que cometeram o erro.

Contexto do papel	Grupo			Erros mais comuns
	Tradicional.	Funções	Animação.	
Uso simples do mais recente titular	10,0	23,5	29,4	Funções e animações: atualização ausente (um)
Uso simples do stepper	40,0	29,4	35,3	Todos: inicialização mal posicionada
Uso complexo do stepper	30,0	52,9	23,5	Funções: atualização ausente (b), ausente inicialização
Uso complexo do mais recente titular	70,0	64,7	41,2	Trad.: utilizou matriz em vez de matriz; Todos: atualização ausente (c)
Uso complexo do coletor	70,0	70,6	64,7	Trad. e anim.: deslocado Inicialização; Funções: ausentes inicialização

Tabela 9. Resumo das origens de erros relacionados a variáveis na construção de programas (E-CONSTR).

Contexto variável	Grupo		
	Tradicional	Funções	Animação
Simples	Estruturas de controle	Estruturas de controle	Estruturas de controle
Complexo	Estruturas de controle	Compreensão do domínio	Estruturas de controle
	Conhecimento de planejamento	Conhecimento de planejamento	

3.6. Discussão

As questões da prova exigiam diversas atividades relacionadas à programação: simulação de programas, compreensão de programas e construção de programas. Até mesmo Embora a maioria das diferenças nos resultados não seja estatisticamente significativa, as tendências sugerem que os efeitos do uso de papéis e da animação baseada em papéis dependem da natureza da atividade.

Na simulação do programa, as diferenças entre os grupos são mínimas em comparação com... em relação aos outros dois tipos de perguntas. Os grupos tradicional e de papéis tiveram desempenhos semelhantes. Bem, embora o desempenho do grupo de animação tenha sido ligeiramente pior. A lógica de O programa a ser simulado era complicado e trabalhoso para os alunos, então A única maneira de descobrir sua saída era simular sua execução cuidadosamente. Mesmo Embora os papéis das variáveis fossem fáceis de encontrar, os momentos em que as variáveis eram As atualizações não eram fáceis de prever. Portanto, é natural que o conhecimento dos papéis... não ajuda nesta tarefa.

Jehng et al. (1999) estudaram os efeitos da visualização na aprendizagem da recursão. Eles encontraram diferenças menores em tarefas em que os participantes tinham que prever o resultado de programas do que em tarefas de construção de programas. Nossos resultados concordam com essa conclusão.

Na compreensão do programa, o grupo de papéis obteve as melhores pontuações, enquanto o O grupo de animação obteve as piores notas. Surpreendentemente, isso também se verificou para as notas. de resumos do programa demonstrando pleno entendimento do programa a ser realizado também compreendido. Uma análise dos resumos do programa revelou que o O grupo de funções forneceu resumos mais detalhados e de baixo nível do que o grupo de animação. Além disso, a proporção de declarações de baixo nível nos resumos dos programas correlacionou-se positivamente com as notas, assim como a proporção de declarações de alto nível. apresentaram correlação negativa. Assim, os professores atribuíram notas melhores às respostas detalhadas. Explicar o funcionamento do programa instrução por instrução é mais adequado do que uma abordagem de alto nível. Resumos que descrevem questões não diretamente visíveis no programa. Como o Os resumos do grupo de animação enfatizaram informações de alto nível, suas notas. estavam baixos.

Informações de alto nível, no entanto, são um indicativo de habilidade superior em programação. (De'tienne, 2002; Pennington, 1987). Por exemplo, Hoadley et al. (1996) demonstraram A reutilização de código — que demonstra habilidade de programação semelhante à de um especialista — foi substancialmente mais comum entre os alunos que incluíram informações de alto nível em seus trabalhos.

resumos de programas. Assim, os professores em nosso experimento atribuíram notas melhores a um nível de compreensão inferior. Pode-se questionar se esse comportamento é comum entre professores de programação, que geralmente desconhecem até mesmo os resultados mais importantes da psicologia da programação.

A natureza geral dos resumos dos programas fornecidos pelos participantes do grupo de animação pode ser explicada pelas diferenças nos softwares utilizados nos exercícios de animação. Uma entrevista semiestruturada com o professor que supervisionou todas as sessões de exercícios dos grupos de interpretação de papéis e animação revelou que os usuários do PlanAni se concentraram mais nas variáveis, enquanto os usuários do depurador passaram a maior parte do tempo acompanhando o código do programa (Sajaniemi & Kuittinen, 2003). Embora o PlanAni mostre cada fragmento de código rapidamente antes de animar seu efeito, os alunos aparentemente não acompanharam o código. Consequentemente, os usuários do depurador obtiveram uma melhor compreensão das ações detalhadas do código, enquanto os usuários do PlanAni obtiveram uma melhor compreensão do efeito total do programa e de como cada variável contribuiu para ele. Isso pode ter afetado a maneira como os usuários do PlanAni pensam sobre os programas: eles podem considerar os ciclos de vida das variáveis mais importantes do que as ações individuais do programa.

A análise dos resumos dos programas revelou também que os grupos de papéis e animação possuíam um vocabulário mais amplo para discutir os programas. Além disso, eles apresentaram predominantemente resumos de programas com referências cruzadas, enquanto o grupo tradicional produziu resumos em nível de domínio e em nível de programa. Essa diferença foi estatisticamente significativa ($p = 0,0046$). Pennington (1987) utilizou um método semelhante para classificar resumos de programas e constatou que os programadores com alta compreensão utilizavam quase que uniformemente resumos com referências cruzadas, enquanto os programadores com baixa compreensão tendiam a produzir resumos em nível de programa ou em nível de domínio. Nossos resultados indicam que o conhecimento de papéis proporciona aos alunos uma estrutura conceitual que eles podem utilizar para processar melhor o conhecimento sobre programas.

Na construção do programa, os grupos de atuação e animação superaram o grupo tradicional. As pontuações do grupo de atuação foram melhores do que as do grupo de animação, mas isso pode ser parcialmente atribuído ao ajuste das notas em pontuações que proporcionou a maior compensação ao grupo de atuação.

A análise de erros revelou que o grupo de animação apresentou menos problemas com variáveis e demonstrou o melhor conhecimento do planejamento. O grupo de animação também obteve o melhor desempenho em relação ao conhecimento profundo do programa, enquanto o grupo tradicional apresentou o pior desempenho. No caso do conhecimento superficial, as diferenças foram opostas, porém pequenas. Segundo Pennington (1987), o conhecimento profundo é um indicativo de melhor compreensão do programa. Podemos, portanto, concluir que o grupo tradicional possuía o modelo mental mais superficial, enquanto o grupo da animação apresentava a melhor representação mental.

As diferenças no nível das representações mentais podem ser explicadas pelo conhecimento de papéis e pelo uso do PlanAni. O conhecimento de papéis faz diferença na posição das variáveis dentro de um programa: na abordagem tradicional, uma variável não tem significado especial por si só, sendo apenas o objeto de algumas atribuições — aparentemente mais ou menos incidentais para um novato. O conhecimento de papéis inverte essa situação: uma variável é um sujeito ativo que se encarrega de alguma tarefa específica e todas as atribuições podem ser vistas como parte integrante dela.

apoiar essa tarefa. Assim, os papéis tornam o conhecimento aprofundado do programa mais acessível aos alunos.

Além disso, o animador de programas PlanAni facilita a assimilação de papéis, atribuindo-lhes uma aparência e um comportamento visuais. Ademais, o histórico visível de valores anteriores (e, por vezes, até futuros) enfatiza a sucessão de valores da variável. Essas características fomentam a assimilação do conhecimento sobre os papéis e, portanto, podem contribuir para uma melhor compreensão das propriedades intrínsecas dos programas.

4. CONCLUSÕES

Realizamos um experimento em sala de aula para estudar os efeitos do conceito de papéis variáveis e da animação baseada em papéis no ensino de programação introdutória. Os alunos foram divididos em três grupos que receberam instruções diferentes: da maneira tradicional, sem o uso de papéis ao longo do curso; utilizando papéis durante todo o curso; e utilizando um animador de programas baseado em papéis, além do uso de papéis no ensino.

Os resultados mostram que os alunos foram capazes de compreender o conceito de papéis e aplicá-lo em novas situações: após o curso, 35% dos participantes usaram nomes de papéis em suas respostas às provas, mesmo que as questões não mencionassem papéis de forma alguma. Mas a exposição aos papéis resultou não apenas em um vocabulário mais rico; um efeito ainda mais importante foi que os papéis proporcionaram aos alunos uma nova estrutura conceitual que os capacitou a processar mentalmente informações de programação de maneira semelhante à de bons leitores de código.

O uso de animações baseadas em papéis pareceu favorecer a assimilação do conhecimento de papéis, visto que os usuários de animações apresentaram menos dificuldades com variáveis na construção de programas. Além disso, esses usuários tenderam a enfatizar estruturas complexas de programas, o que indica uma melhor compreensão. Portanto, tanto o uso de papéis no ensino quanto o uso de animações baseadas em papéis levaram a resultados que indicam melhores habilidades de programação.

A compreensão mais profunda dos usuários do animador, no entanto, não se refletiu nas notas do curso. Os professores atribuíram notas melhores para resumos detalhados da estrutura superficial do que para respostas que revelavam uma compreensão profunda. Seria interessante verificar se esse comportamento é comum entre professores cujo conhecimento da psicologia da programação geralmente é limitado. Além disso, essa contradição entre notas e habilidades de programação significa que a avaliação de experimentos em sala de aula não deve se basear em uma simples atribuição de notas, mas em uma análise cuidadosa dos modelos mentais resultantes.

Na simulação de programas, o uso de papéis não teve efeito, embora isso possa ser simplesmente um problema de avaliação: o formato das respostas à questão da simulação não forneceu meios para analisar os modelos mentais dos alunos. Tanto na compreensão quanto na construção de programas, foram obtidos resultados positivos. No entanto, é impossível comparar a magnitude do efeito nesses dois casos, principalmente porque a literatura sobre a avaliação de modelos mentais na construção de programas é escassa em comparação com a literatura sobre a compreensão de programas.

Em nosso experimento, todos os grupos experimentais receberam basicamente o mesmo ensino, baseado em um curso já existente. Os papéis foram apresentados aos dois grupos experimentais durante o curso, quando surgiram pela primeira vez no exemplo.

programas. Seria interessante verificar se a adaptação do curso para melhor atender ao conceito de função aumentaria os efeitos descritos acima.

AGRADECIMENTOS

Os autores gostariam de agradecer a Elina Räsänen, Markku Hauta-Kasari, Jenni Pitkänen e Matti Niemi por atuarem como professores do curso; Pauli Byckling, Pauli Harjumaaki e Veli-Pekka Laasonen pela ajuda prática na realização do experimento.

REFERÊNCIAS

- Ben-Ari, M., & Sajaniemi, J. (2004). Papéis das variáveis na perspectiva dos educadores de ciência da computação. In Anais da 9ª Conferência Anual sobre Inovação e Tecnologia no Ensino de Ciência da Computação (pp. 52 – 56). Nova York: ACM Press.
- Clancy, MJ, & Linn, MC (1999). Padrões e pedagogia. In Anais do 30º Simpósio Técnico SIGCSE sobre Educação em Ciência da Computação. Boletim ACM SIGCSE, 31, 37 – 42.
- Davies, SP (1993). Modelos e teorias de estratégia de programação. Revista Internacional de Man-Estudos de Máquinas, 39, 237 – 267.
- Davies, SP (1996). Estratégias de resolução de problemas baseadas em exibição na programação de computadores. Em WD Gray & DA Boehm-Davis (Eds.), Estudos Empíricos de Programadores: Sexto Workshop. Norwood, NJ: Ablex Publishing Company.
- De'ienne, F. (2002). Design de Software—Aspectos Cognitivos. Londres: Springer-Verlag.
- Ehrlich, K., & Soloway, E. (1984). Uma investigação empírica do conhecimento tácito do plano em programação. Em JC Thomas & ML Schneider (Eds.), Fatores Humanos em Sistemas de Computadores, (pp. 113 – 133). Norwood, NJ: Ablex Publishing Company.
- Feldman, M. (1999). Ada95 Problem Solving and Program Design. Reading, MA: Addison-Wesley.
- Fincher, S. (1999). O que estamos fazendo quando ensinamos programação? In Anais da 29ª Conferência ASEE/IEEE Frontiers in Education, (pp. 1 – 5), San Juan, Porto Rico. Sessão 12a4.
- Fincher, S., & Utting, I. (2002). Padrões pedagógicos: seu lugar no gênero. In Anais da 7ª Conferência Anual sobre Inovação e Tecnologia no Ensino de Ciência da Computação. Nova York: ACM Press.
- Fleury, A. (1997). Executando algoritmos: como e por que funciona. In Anais do 4º Anual Conferência Centro-Oeste da CCSC, Universidade Dominicana.
- Ginat, D. (2001). Eficiência inicial de algoritmos com padrões de projeto. Educação em Ciência da Computação, 11, 89 – 109.
- Good, J. (1999). Paradigmas de Programação, Tipos de Informação e Representações Gráficas: Investigações Empíricas da Compreensão de Programas por Iniciantes. Tese de doutorado, Universidade de Edimburgo.
- Green, TRG, & Cornah, AJ (1985). A tocha do programador. Em Interação Humano-Computador - INTERACT'84, (pp. 397 – 402). IFIP, Elsevier Science Publishers (North-Holland).
- Hagan, D., & Sheard, J. (1998). O valor das aulas de discussão para o ensino de programação introdutória. In Anais da 3ª Conferência Anual sobre Inovação e Tecnologia no Ensino de Ciência da Computação (pp. 108 – 111). Nova York: ACM Press.
- Hanly, JR, & Koffman, EB (1999). Resolução de problemas e projeto de programas em C. Reading, MA: Addison-Wesley.
- Hoadley, CH, Linn, MC, Mann, LM, & Clancy, MJ (1996). Quando, por que e como programadores novatos reutilizam código? Em WD Gray & DA Boehm-Davis (Eds.), Estudos Empíricos de Programadores: Sexto Workshop (pp. 109 – 129). Norwood, NJ: Ablex Publishing Corporation.

82 J. Sajaniemi e M. Kuittinen

- Hundhausen, CD, Douglas, SA, & Stasko, JD (2002). Um metaestudo da visualização de algoritmos. eficácia. *Revista de Linguagens Visuais e Computação*, 13, 259 – 290.
- Jehng, J.-CJ, Tung, S.-HS, & Chang, C.-T. (1999). Uma abordagem de visualização para aprender o conceito de recursão. *Journal of Computer Assisted Learning*, 15, 279 – 290.
- Jenkins, T. (1998). Uma abordagem participativa para o ensino de programação. In *Anais da 3ª Conferência Anual sobre Inovação e Tecnologia no Ensino de Ciência da Computação* (pp. 125 – 129). Nova Iorque: ACM Press.
- Koffman, EB (1986). *Turbo Pascal: Uma abordagem de resolução de problemas*. Reading, MA: Addison-Wesley.
- McKay, E. (1999a). Explorando o efeito de metáforas gráficas no desempenho da aprendizagem de conceitos de programação de computadores em alunos adultos: Um estudo piloto. *Psicologia Educacional*, 19, 471 – 487.
- McKay, E. (1999b). Uma investigação de materiais instrucionais baseados em texto aprimorados com gráficos. *Psicologia Educacional*, 19, 323 – 335.
- Nguyen, D. (1998). Padrões de projeto para estruturas de dados. In *Anais da 29ª Conferência Técnica da SIGCSE Símpósio sobre Educação em Ciência da Computação*. *Boletim ACM SIGCSE*, 30, 336 – 340.
- Pennington, N. (1987). Estratégias de compreensão em programação. Em GM Olson, S. Sheppard & E. Soloway (Eds.), *Estudos empíricos de programadores: Segundo workshop* (pp. 100 – 113). Norwood, NJ: Ablex Publishing Company.
- Perkins, DN, & Martin, F. (1986). Conhecimento frágil e estratégias negligenciadas em programadores novatos. Em E. Soloway & S. Iyengar (Eds.), *Estudos Empíricos de Programadores* (pp. 213 – 229). Norwood, NJ: Ablex Publishing Company.
- Pirolli, PL, & Anderson, JR (1985). O papel da aprendizagem por exemplos na aquisição de habilidades de programação recursiva. *Canadian Journal of Psychology*, 39, 240 – 272.
- Rist, RS (1989). Criação de esquemas em programação. *Ciência Cognitiva*, 13, 389 – 414.
- Sajaniemi, J. (2002). Uma análise empírica dos papéis das variáveis em programas procedimentais de nível iniciante. Em *Anais do Simpósio IEEE de 2002 sobre Linguagens e Ambientes de Computação Centrados no Ser Humano (HCC'02)* (pp. 37 – 39). IEEE Computer Society.
- Sajaniemi, J. (2003). Página inicial sobre os papéis das variáveis. (http://www.cs.joensuu.fi/~saja/var_roles/).
- Sajaniemi, J., & Kuittinen, M. (2003). Animação de programas baseada nos papéis das variáveis. In *Anais do Simpósio ACM 2003 sobre Visualização de Software* (pp. 7 – 16). Nova York: ACM Press.
- Wallingford, E. (2003). Página inicial de padrões elementares. (<http://www.cs.uni.edu/~wallingf/padrões/elementares/>).
- Wiedenbeck, S. (1989). Aprendendo iteração e recursão a partir de exemplos. *Revista Internacional de Estudos Homem-Máquina*, 30, 1 – 22.
- Wilson, BC, & Shrock, S. (2001). Contribuindo para o sucesso em um curso introdutório de ciência da computação: um estudo de doze fatores. In *Anais do 32º Simpósio Técnico SIGCSE sobre Educação em Ciência da Computação*. *Boletim ACM SIGCSE*, 33, 184 – 188.